思考题 11 16

一、问题分析

城市街道是 **树形结构**，在一些路口设立分部。

要求：**相邻路口不能同时设分部**（避免竞争）。

目标：

1. 若每个分部利润都一样，怎样布点才能让总利润最大？
2. 若每个路口的利润不同，又该怎么布点？

抽象成图论问题：

- 每个路口 = 一个结点；
- 每条街道 = 一条边；
- “街道结构为树形” = 这张图是一棵树（连通、无环）；
- “相邻街道不重复设置分部” = 每条边两端不能同时选；

  这就是在树上选一些 **不相邻的结点**，使得：
- 若利润相同：选的结点**个数最多**；
- 若利润不同：选的结点**权值和最大**。

即为：

树上的最大独立集（Maximum Independent Set on Tree）

（每个结点有权值时就是“带权最大独立集”）。

二、算法思路（树形 DP）

树形结构**没有环，可以从根往下做动态规划**。

1. 设计状态

对树的每个结点 u，定义两个状态：

- `dp[u][0]`：在以 u 为根的子树里，**不选**结点 u 时，这棵子树能得到的最大利润；

- `dp[u][1]`：在以 u 为根的子树里，**选了**结点 u 时，这棵子树能得到的最大利润。

假设结点 u 的利润是 `w[u]`

（如果所有结点利润相同，那就是第一种情况）。

2. 状态转移

对结点 u 的每个“子结点” v（即跟 u 相邻、但在 DFS 中当作孩子的结点）：

1. **如果选了结点 u**（状态 1）

   那 u 的所有孩子 v 就 **不能再选**，否则相邻冲突。

   因此：

$$dp[u][1] = w[u] + \sum_{v\ 是u的孩子} dp[v][0]$$

2. **如果不选结点 u**（状态 0）

   每个孩子 v 可以选也可以不选，选更优的那个：

$$dp[u][0] = \sum_{v\ 是u的孩子} \max(dp[v][0],\ dp[v][1])$$

   整棵树处理完后（随便选一个结点，比如 1 号点为根），答案就是：

$$\text{ans} = \max(dp[1][0],\ dp[1][1])$$

- 若所有 `w[u] = 1`，就是 **“最多选多少个结点”**，即第一种情况；
- 若 `w[u]` 各不相同，就是 **“最大总利润”**，即第二种情况。

3. 遍历顺序

因为是树：

1. 把树存成邻接表；
2. 从某个点（如 1 号）开始 DFS；
3. 在 DFS 里一边递归，一边根据孩子的 `dp` 来算父亲的 `dp`。

DFS 参数里带一个 `parent`，避免从孩子又走回父亲产生死循环。

三、C 语言完整代码（兼容“利润相同”和“利润不同”）

```c
#include <stdio.h>
#include <string.h>
#define MAXN 100005   // 最大结点数
// 邻接表存树
int head[MAXN];
int to[2 * MAXN];       // 边的终点
int nxt[2 * MAXN];      // 下一条边的编号
int idx = 0;           // 边计数器
int w[MAXN];    // 每个结点的利润
int dp[MAXN][2]; // dp[u][0], dp[u][1]
int N;
// 添加一条无向边 u - v
void add_edge(int u, int v) {
    to[idx] = v;
    nxt[idx] = head[u];
    head[u] = idx;
    idx++;
}
void dfs(int u, int parent) {
    // 初始化：如果选 u，至少得到自己的利润；不选 u，初始为 0
    dp[u][1] = w[u];
    dp[u][0] = 0;
    // 遍历所有邻居 v
    for (int e = head[u]; e != -1; e = nxt[e]) {
        int v = to[e];
        if (v == parent) continue; // 防止走回去
        dfs(v, u); // 先计算子树 v 的 dp
        // 不选 u
        if (dp[v][0] > dp[v][1])
            dp[u][0] += dp[v][0];
        else
            dp[u][0] += dp[v][1];
        // 选 u
        dp[u][1] += dp[v][0];
    }
}
```

```c
int main() {
    // 初始化 head 数组为 -1
    memset(head, -1, sizeof(head));
    printf("请输入结点个数：");
    scanf("%d", &N);
    printf("请按顺序输入结点利润：");
    for (int i = 1; i <= N; i++) {
        scanf("%d", &w[i]);   // 利润，可以都输 1 表示“利润相同”
    }
    printf("请输入边：\n");
    for (int i = 0; i < N - 1; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        add_edge(u, v);
        add_edge(v, u);
    }
    // 以 1 号结点为根做 DFS
    dfs(1, 0);
    int ans = dp[1][0] > dp[1][1] ? dp[1][0] : dp[1][1];
    printf("最大利润：d\n", ans);
    return 0;
}

```

四、运行示例

1. 如果是“利润不同”的情况

假设城市是这样一棵树（5 个路口）：

- 边：1-2, 1-3, 3-4, 3-5
- 各点利润（不相同的情况示例）

```
请输入结点个数： 5
请按顺序输入结点利润： 5 3 4 2 1
请输入边：
1 2
1 3
3 4
3 5
最大利润： 8
```

：

2. 如果是“利润相同”的情况

```
请输入结点个数： 5
请按顺序输入结点利润： 1 1 1 1 1
请输入边：
1 2
1 3
3 4
3 5
最大利润： 3
```

五、时间复杂度

- 每条边只被访问两次（无向图：u->v, v->u）。
- 每个结点只被计算一次状态。

时间复杂度：**O(N)**，空间复杂度也约为 **O(N)**。

N 为路口数量（结点数）。

六、方法对比

- **贪心 1**：从度数最小的路口开始选；
- **贪心 2**：总是选当前利润最大的点、并删除它的邻居；

这些方法在一般图/树上都不保证最优，很容易构造反例。

而上面给出的 **树形 DP**：

- 每条边只处理一次；
- 严格按状态转移推理；
- 能保证给出全局最优解。

一、问题分析

题意：

- 有 n 张魔法卡，第 i 张卡的价值为 arr[i]（正整数）。
- 现在要把这些卡 **分给两个孩子 A 和 B**。

- 目标：**让两个人拿到的总价值尽可能接近（差最小）**。

分析：

> 给定若干正整数，把它们分成两个子集，使得两个子集元素和的差值最小。

设所有卡的总价值为 `S`，

给孩子 A 一些卡，总价值 `S1`；

则 B 得到的就是 `S2 = S - S1`。

要最小化，则:

$$|S1 - S2| = |S1 - (S - S1)| = |2S1 - S|$$

所以问题等价于：

> 在所有卡中选出一些卡，使得它们的总和 `S1` **不超过 S/2**，
> 并且尽可能 **接近 S/2（最大化 S1）**。

二、算法思路：0/1 背包式动态规划

1. 状态设计

设总和 `S = a[1] + a[2] + ... + a[n]`

目标容量 `T = S / 2`（向下取整）。

定义一维 DP 数组：//优化二维数组为一维动态数组

- `dp[j]`：在**容量为 j** 的限制下，使用前若干张卡，
  能得到的 **最大总价值**（不超过 j）。

初始化：

- `dp[0] = 0`
- 其他 `dp[j]` 初始也是 0（表示用不了任何卡时最大和为 0）

2. 状态转移（0/1 背包）

遍历每张卡 `i`，价值 `a[i]`，然后从右向左更新 `dp`：

```
for i = 1..n
  for j = T down to a[i]
    dp[j] = max(dp[j], dp[j - a[i]] + a[i])
```

解释：

- `dp[j]`：当前不拿第 `i` 张卡；
- `dp[j - a[i]] + a[i]`：拿第 `i` 张卡（前面容量只剩 j-a[i]）。

右向左遍历是为了保证 **每张卡最多用一次**（0/1 背包）。

3. 最终答案

循环结束后，`dp[T]` 就是 **不超过 S/2 的最大总价值 S1**：

- A 的总价值 `S1 = dp[T]`
- B 的总价值 `S2 = S - S1`
- 两者差值 `diff = (S - S1) - S1 = S - 2 * S1`

三、C 语言完整代码

```c
#include <stdio.h>
#include <stdlib.h>
int maxw(int n, int m) {
    return n > m ? n : m;
}
int main() {
    int n;
    printf("请输入魔卡数量：");
    scanf_s("%d", &n);
    int* arr = (int*)malloc(sizeof(int) * (n+1));
    int sum = 0;
    printf("请输入魔卡价值：");
        for (int i = 1; i <= n; i++) {
            scanf_s("%d", &arr[i]);
            sum += arr[i];
    }
        int T = (int)(sum / 2);//使目标尽量接近sum/2
        int* dp = (int*)malloc(sizeof(int) * (T + 1));
        for (int j = 0; j <= T; j++) {//初始化dp数组
            dp[j] = 0;
        }
        for (int i = 1; i <= n; i++) {
            for (int j = T; j >= arr[i]; j--) {
                dp[j] = maxw(dp[j], dp[j - arr[i]] + arr[i]);
            }
        }
        int S1 = dp[T];
        int S2 = sum - S1;
        int diff = sum - (S1 + S2);
        printf("ChildA:%d\n", S1);
        printf("ChildB:%d\n", S2);
        printf("Difference:%d\n", diff);
        free(arr);
        free(dp);
        return 0;
}
```

四、运行示例

```
请输入魔卡数量：5
请输入魔卡价值：3 1 4 2 2
ChildA:6
ChildB:6
Difference:0
```

五、时间 / 空间复杂度

- 总价值为 S，目标容量 `T = S/2`
- 时间复杂度：`O(n * T)`
- 空间复杂度：`O(T)`（一维 DP）；如果是二维数组，则为 `O(n * T)`

六、优化

具体拿哪些卡？

- 现在程序只算了“最大和”，没告诉具体选了哪些卡。
- 想恢复方案，可以：
  - 用二维 `dp[i][j]`，记录用前 i 张卡时的结果；
  - 再从 `dp[n][T]` 往前回溯：比较 `dp[i][j]` 与 `dp[i-1][j]` 是否相等，

    不相等说明第 i 张卡被选了。
- 这样就可以真正列出 A 和 B 分别拿哪些卡。